

CSD 3131 Algorithm Analysis



Team 12

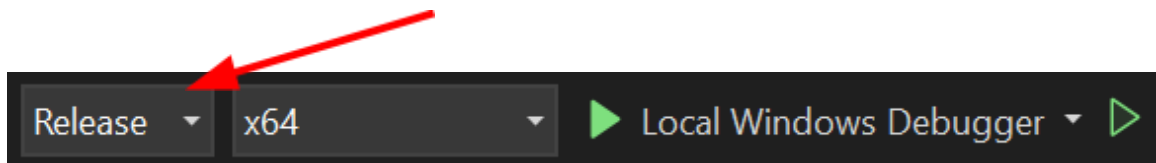
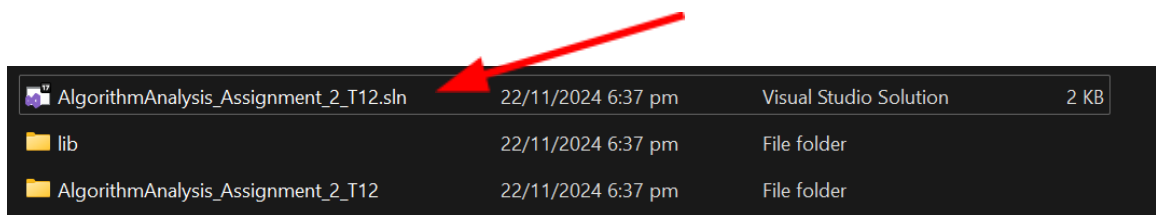
<u>Team members:</u>	<u>Student ID:</u>
Clifford Tan	2201284
George Koh	2201598
Solomon Ng	2200713
Ricky Kwong	2201238
Ong Junkang	2201187
Zhuo Yuanqin	2202183

Content Page

1. Project Guide.....	3
2. Seam Carving (Dynamic Programming).....	4
3. Seam Carving Analysis.....	9
4. Seam Carving (Greedy Algorithm).....	11
5. Overall Analysis and Discussion.....	17
6. Additional Features.....	19
7. References.....	30

1. Project Guide

- Unzip the AlgorithmAnalysis_Assignment_2_T12.zip file
- Go into the AlgorithmAnalysis_Assignment_2_T12 folder
- Click on the AlgorithmAnalysis_Assignment_2_T12.sln
- And run the solution (.sln) preferably in Release Mode for a smoother experience
- The fonts and images are stored in the assets folder located in the AlgorithmAnalysis_Assignment_2_T12 folder which is the same directory as the solution
- Libraries used in this project are located in the lib folder which is also in the same directory as the solution



2. Seam Carving (Dynamic Programming)

Question 1

Seam carving using Dynamic programming

(Vertical and Horizontal seams)

a) Calculate the Energy Map:

In the function `CalculateEnergyMap`, the function takes in a vector of channels and returns a single matrix containing the calculated energy map. For each channel, it calculates the horizontal gradient (x - direction) using Sobel operator (1, 0, 3) and calculates the vertical gradient (y - direction) using Sobel operator (0, 1, 3). The (1, 0, 3) parameter represents 1st order x derivative, 0th order y derivative in a 3 x 3 kernel. While the (0, 1, 3) parameter represents 0th order x derivative, 1st order y derivative in a 3 x 3 kernel. After calculating both horizontal and vertical gradients, we add the absolute values of both gradients to the energy map. Higher values in the calculations of the energy map indicates that the area of the image has more dramatic changes in colour/intensity.

b) Use Dynamic Programming to Find the Minimum Seam:

Vertical Seam

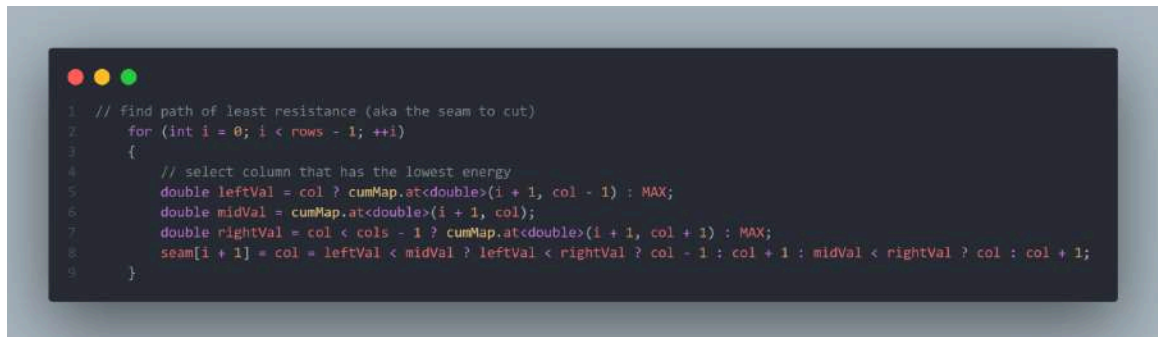
The function `CalculateVerticalCumMap` generates the cumulative map based on the energy map. It goes bottom-up and begins by going through the second last row to find the lowest possible cost to reach each column.

Energy Map						Cumulative Map					
7	9	8	5	3	2	12	14	13	12	11	8
4	3	5	6	2	3	9	5	7	8	6	10
6	5	2	4	7	8	6	5	2	4	7	8

Figure illustrating the cumulative map generated based on an energy map

For example, to reach the column of value 4 in the second last row of the energy map, the sum of possible paths to it is 10 (4+6) and 9 (5+4). Thus the value stored in the cumulative map for the corresponding column is 9. The process is repeated for the remaining rows.

In the function `FindVerticalSeamDP`, the function takes in a cumulative energy map matrix where each cell contains the cumulative minimum energy up to that point which guides the seam finding process. This function returns a vector of column indices representing the vertical seam path from top to bottom.



```
1 // find path of least resistance (aka the seam to cut)
2 for (int i = 0; i < rows - 1; ++i)
3 {
4     // select column that has the lowest energy
5     double leftVal = col ? cumMap.at<double>(i + 1, col - 1) : MAX;
6     double midVal = cumMap.at<double>(i + 1, col);
7     double rightVal = col < cols - 1 ? cumMap.at<double>(i + 1, col + 1) : MAX;
8     seam[i + 1] = col = leftVal < midVal ? leftVal < rightVal ? col - 1 : col + 1 : midVal < rightVal ? col : col + 1;
9 }
```

This function utilises the hallmarks of dynamic programming where it has Overlapping subproblems and Optimal substructure. The implementation stores pre - computed cumulative energies to make locally optimal choices that lead to a globally optimal seam. The function starts by finding the column with minimum cumulative energy in the 1st row and then for each pixel in the subsequent rows, it considers the cumulative energy of only 3 possible directions in its neighbouring columns: Left - Down (`leftVal`), Middle - Down (`midVal`) and Right - Down (`rightVal`). It then selects the direction with the minimum energy and the process repeats until the seam path is traced to the bottom row. We also accounted for boundary handling for edge cases like the leftmost and rightmost columns where for the leftmost column only the middle and right are considered, whereas for the rightmost column only the left and middle are considered. This ensured that there is no out of bounds access.

Horizontal Seam

The function `CalculateHorizontalCumMap` generates the cumulative map based on the energy map. It begins by going through the second last column to find the lowest possible cost to reach each row.

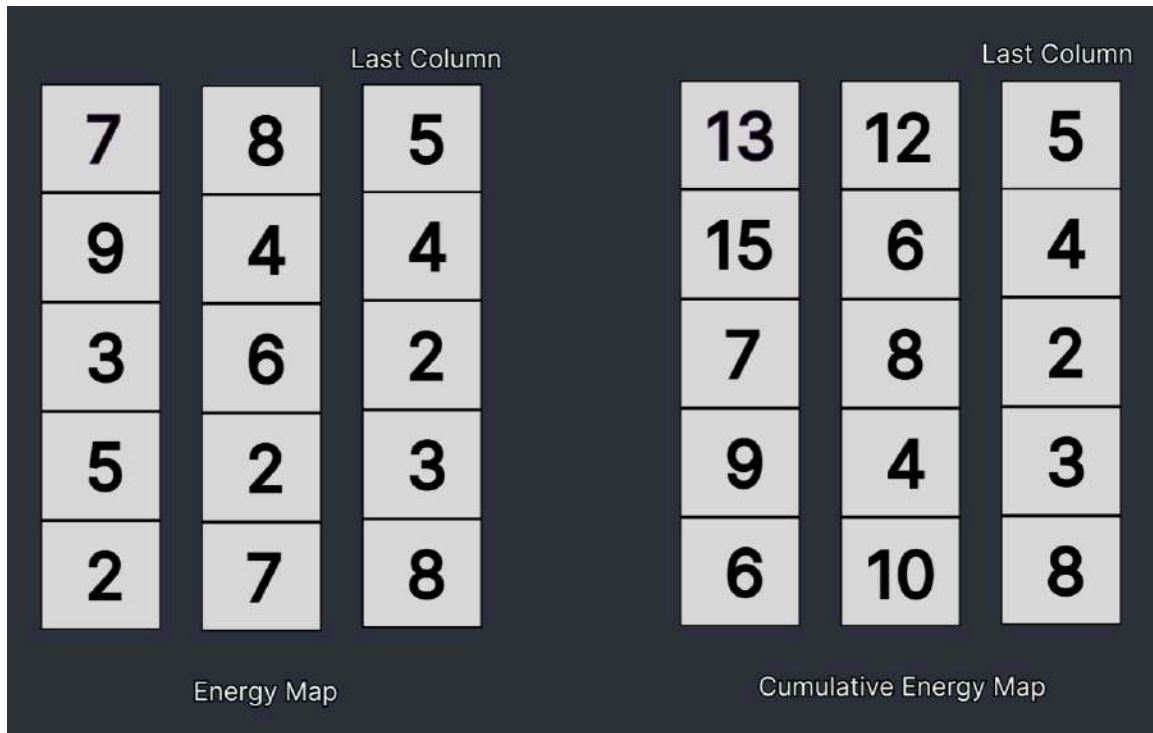


Figure illustrating the cumulative map generated based on an energy map

For example, to reach the column of value 4 in the second last column of the energy map, the sum of possible paths to it from the last column is 9 (4+5), 8 (4+4) and 6 (4+2). Thus the value stored in the cumulative map for the corresponding column is 6. The process is repeated for the remaining rows.

```

1 // find path of least resistance (aka the seam to cut)
2 for (int i = 0; i < cols - 1; ++i)
3 {
4     // select column that has the lowest energy
5     double leftVal = row ? cumMap.at<double>(row - 1, i + 1) : MAX;
6     double midVal = cumMap.at<double>(row, i + 1);
7     double rightVal = row < rows - 1 ? cumMap.at<double>(row + 1, i + 1) : MAX;
8     seam[i + 1] = row = leftVal < midVal ? leftVal < rightVal ? row - 1 : row + 1 : midVal < rightVal ? row : row + 1;
9 }

```

Similarly, for the FindHorizontalSeamDP function, it follows the same dynamic programming approach as the vertical version, but operates from left to right. Instead of columns, it finds the row with the minimum cumulative energy in the first column and traces the seam path through the lowest energy direction (Up-Right, Middle-Right, Down-Right) until reaching the last column. Boundary handling is applied for the topmost

and bottommost rows to avoid out-of-bounds access. The result is a vector of row indices representing the horizontal seam.

c) Remove the Seam and Repeat:

In the function `RemoveVerticalSeam`, the function takes in an image matrix and a vector column indices which represents the seam to be removed. This function modifies the input image and does not have any return value. The function starts by processing each row of the image, and gets the column position of the seam in the current row. After the seam, it shifts all pixels one position leftwards, this removes the seam pixel by overwriting it. Lastly after shifting the pixels, the function resizes the image by reducing the image width by 1 column using `img.colRange(0, cols - 1)`.

Similarly, for the `RemoveHorizontalSeam` function, it modifies the input image by removing the specified horizontal seam. It processes each column to determine the row position of the seam, shifts all pixels below it one position upwards to overwrite the seam pixel, and then resizes the image by reducing its height by one row using `img.rowRange(0, rows - 1)`.

The function `SeamCarvingToWidthDP` and `SeamCarvingToHeightDP` is where the entire seam carving process is being encapsulated.

For Each Iteration:

1. Split the image into its colour channels using `cv::split`
2. Calculate the energy map for the current state of the image with `CalculateEnergyMap`.
3. Compute the cumulative energy with `CalculateCumMap`.
4. Using Dynamic Programming approach to identify the optimal seam with `FindVerticalSeamDP` and `FindHorizontalSeamDP`.
5. Identified seam is removed with `RemoveVerticalSeam` and `RemoveHorizontalSeam`.

The seam carving process implemented is an iterative approach that will continue until the desired image dimensions are achieved.

3. Seam Carving Analysis

Question 2

Given a colour picture consisting of an $m \times n$ array, $\text{img}[1\dots m, 1\dots n]$, of pixels, where each pixel specifies a triple of red, green, and blue (RGB) intensities. Specifically, we wish to remove one pixel from each of the m rows, so that the whole picture becomes one pixel narrower. To avoid disturbing visual effects, however, we require that the pixels removed in two adjacent rows be in the same or adjacent columns; the pixels removed form a “seam” from the top row to the bottom row where successive pixels in the seam are adjacent vertically or diagonally.

a) Show that the number of such possible seams grows at least exponentially in m , assuming that $n > 1$.

To show that the number of possible seams grows at least exponentially with the image height m where $n > 1$, we will be taking the approach of analysing how the seams are constructed. Starting at any pixel in the top row, we need to trace a path to the bottom row where each subsequent pixel must be adjacent. To be considered adjacent, it must be directly below or diagonally connected, we only consider these pixels as the seam will only move downwards when we reconstruct it. For any pixel in the current row, we have up to 3 possible choices for the next pixel in the row below: we can move directly down, diagonally down-left, or diagonally down-right except for pixels on the 1st and n^{th} column which are on the edges of the image.

Let's consider the most restrictive cases where the pixels are on the edges of the image, we can prove that each pixel still has at least 2 possible choices for the next pixel:

1. Pixels in the 1st column ($j = 1$): can move down or down-right
2. Pixels in the last column ($j = n$): can move down or down-left
3. Any pixels other than those in the 1st & last column: can move down, down-left or down-right

Since $n > 1$, we are guaranteed to have at least 2 choices at each step. As we construct the seam from top to bottom, we make $m - 1$ consecutive choices where each choice is considered between each pair of adjacent rows. Thus we have at least 2 choices at each of these $m - 1$ steps, the total number of possible seams would then be at least 2^{m-1} . Since 2^{m-1} grows exponentially with m , this successfully proves that the number of possible seams grows at least exponentially with the image height.

b) Suppose now that along with each pixel $\text{img}[i, j]$, we have calculated a real valued energy or disruption measure $e[i, j]$, indicating how disruptive or energy effective it would

be to remove pixel $\text{img}[i,j]$. Intuitively, the lower a pixel's disruption measure (energy), the more similar the pixel is to its neighbours. Suppose further that we define the disruption measure of a seam to be the sum of the disruption measures of its pixels. In other words, how to relate to subproblems?

Given a colour picture consisting of an $m \times n$ array $\text{img}[1 \dots m, 1 \dots n]$, let each pixel $\text{img}[i,j]$ have a disruption measure $e[i,j]$, where lower values indicate pixels that are more similar to their neighbours and hence less disruptive to remove. Our objective is to find a seam—a connected sequence of pixels from the top to the bottom row—such that the total disruption measure of the seam is minimised.

To approach this Question, we define a subproblem: Let $\text{dp}[i][j]$ represent the minimum disruption measure of a seam that ends at pixel (i, j) . Then $\text{dp}[i][j]$ can be computed as follows:

1. Initialization: For the first row, we set $\text{dp}[1][j] = e[1][j]$ for each pixel j in the range $1 \leq j \leq n$, as each pixel in the top row represents a seam that consists of just that one pixel.
2. Recursive Relation: For each subsequent row i , the minimum disruption measure of a seam ending at pixel (i, j) is the disruption of $\text{img}[i, j]$ plus the minimum disruption measure of the seam ending at one of the pixels in the previous row that are adjacent to (i, j) (directly above, above-left, or above-right).

$$\text{Thus: } \text{dp}[i][j] = e[i][j] + \min(\text{dp}[i-1][j-1], \text{dp}[i-1][j], \text{dp}[i-1][j+1])$$

Here, $\text{dp}[i-1][j-1]$, $\text{dp}[i-1][j]$, and $\text{dp}[i-1][j+1]$ represent the minimum disruption measure for seams ending at the three possible adjacent pixels in the previous row, with boundaries handled for the leftmost and rightmost columns.

3. Final Solution: Once we reach the last row, the minimum-disruption seam value is the smallest entry in $\text{dp}[m][j]$ for $j=1, \dots, n$: $\text{min_energy_seam} = \min(\text{dp}[m][j])$

This recursive breakdown allows us to efficiently calculate the minimum disruption seam by focusing on the relationship between each pixel and its adjacent pixels in the row above, thereby reducing the problem to manageable subproblems and building up to the final solution with minimal disruption.

4. Seam Carving (Greedy Algorithm)

Question 3

Seam carving using Greedy algorithm

(Vertical and Horizontal seams)

The function `FindVerticalSeamGreedy`, takes in a cumulative energy map matrix that was calculated [earlier](#). Similar to the dynamic programming variant, this function also returns a vector of column indices that represents the vertical seam from the last row of the energy map to the first row.

Since the function uses the Greedy algorithm, the function goes through all columns in the last or first row of the energy map (excluding edges) to find the column with the lowest energy value, then stores the column index of the minimum location in the vector seam.

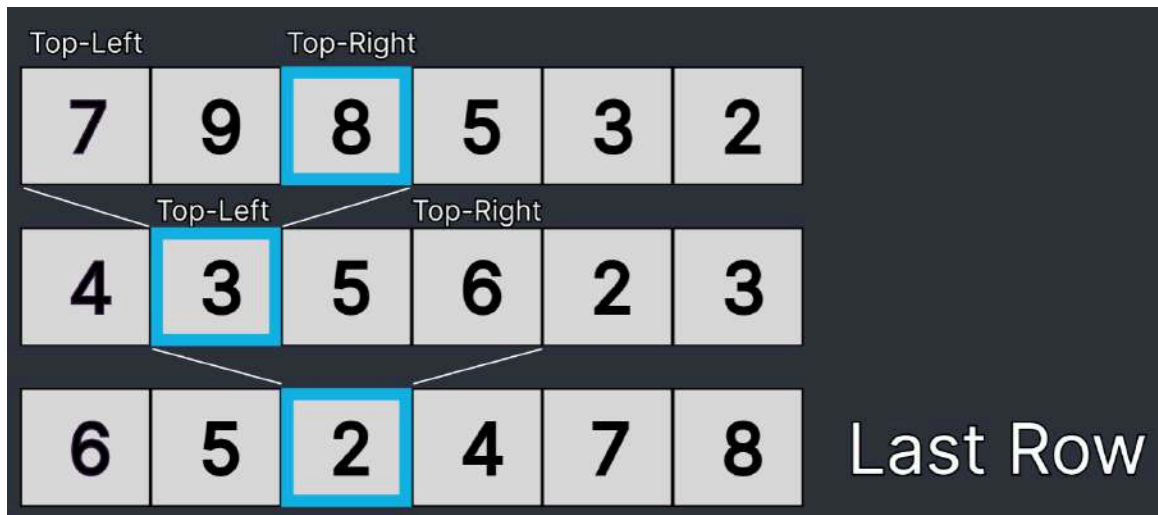


Figure illustrating how the vertical seam is generated using Greedy algorithm

As seen in the example above, 2 is the lowest energy value in the last row, thus the index for that specific column will be stored. Then, it checks the column directly above, top-left (`startCol`), and top-right (`endCol`) for the minimum value of the 3 columns. Between 3, 5 and 6, 3 has the lowest value thus the index for that column will be stored in seam. The process of checking the column directly above, top-left and top-right for the minimum value of the 3 columns and storing the index is repeated for the remaining rows.

Boundary handling for cases when the minimum location is at edges (leftmost column of the range or rightmost column of the range) is also accounted for.

```

1 // get start and end col (directly below, left below or right below), (ignore edges)
2 int startCol = std::max(1, prevCol - 1);
3 int endCol = std::min(cols - 2, prevCol + 1);
4
5 // find minimum energy value in the neighboring columns
6 cv::minMaxLoc(energyMap.row(row).colRange(startCol, endCol), &minVal, nullptr, &minLoc);

```

After getting the location of the minimum for all rows, the vector, seam, containing all the indices is returned.

The function `FindHorizontalSeamGreedy` is similar to `FindVerticalSeamGreedy` in that both functions take in an energy map matrix. However, instead of returning a vector of column indices that represents the vertical seam, it returns a vector of row indices that represents the horizontal seam.

Similarly, since it also uses the Greedy algorithm, the function goes through all rows in the last column of the energy map (also excluding edges) to find the row with the lowest energy value, then stores the row index of the minimum location in the vector, seam.

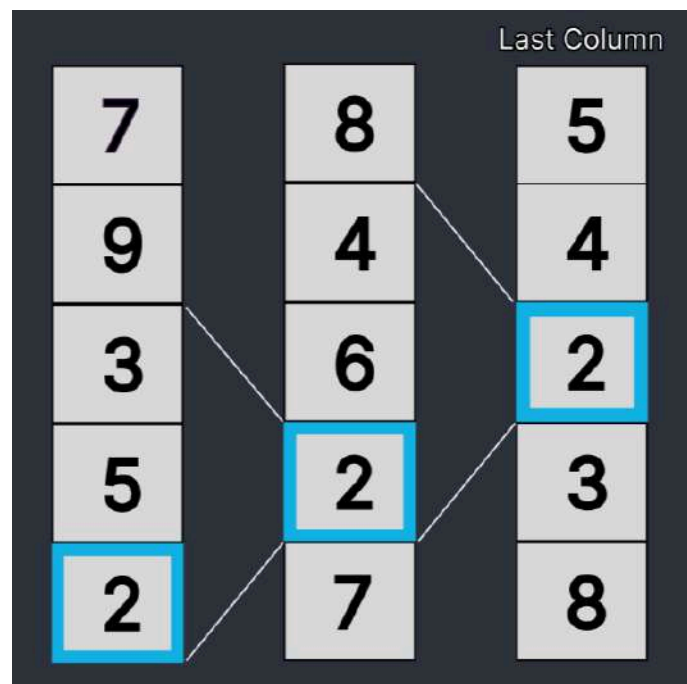


Figure illustrating how the Horizontal seam is generated using Greedy algorithm

As seen in the example above, 2 is the lowest energy value in the last column, thus the index for that specific row will be stored. Then, it checks the row next to it, top-left

(startRow), and bottom-left (endRow) for the minimum value of the 3 rows. Between 2, 4 and 6, 2 has the lowest value thus the index for that row will be stored in seam. The process of checking the row next to it, top-left and bottom-left for the minimum value of the 3 rows and storing the index is repeated for the remaining columns.

Boundary handling for cases when the minimum location is at edges (lowest row of the range or highest row of the range) is also accounted for.

```
1 // iterate through all columns to find horizontal seam
2 for (int col = cols - 2; col >= 0; --col)
3 {
4     // get previous row
5     int prevRow = seam[col + 1];
6
7     // get start and end row (directly below, left below or right below), (ignore edges)
8     int startRow = std::max(1, prevRow - 1);
9     int endRow = std::min(rows - 2, prevRow + 1);
10
11     // find minimum energy value in the neighboring columns
12     cv::minMaxLoc(energyMap.col(col).rowRange(startRow, endRow), &minVal, nullptr, &minLoc);
13
14     seam[col] = startRow + minLoc.y; // adjust seam index based on the range
15 }
```

After getting the location of the minimum for all columns, the vector, seam, containing all the indices is returned.

Both functions FindVerticalSeamGreedy and FindHorizontalSeamGreedy finds the seam from the last row and column (excluding edges) respectively. However, since both functions are using the Greedy algorithm, the results may vary if the seam is created from the first row and first column respectively.

In both the functions VerticalSeamCarvingGreedy and HorizontalSeamCarvingGreedy, the entire seam carving process is being encapsulated.

Similar to the Dynamic Programming variant, for each iteration:

1. Split the image into its colour channels using `cv::split`
2. Calculate the energy map for the current state of the image with `CalculateEnergyMap`.
3. Compute the cumulative energy with `CalculateCumMap` and .
4. Using Dynamic Programming approach to identify the optimal seam with `FindVerticalSeamGreedy` and `FindHorizontalSeamGreedy`.

5. Identified seam is removed with `RemoveVerticalSeam` and `RemoveHorizontalSeam`.

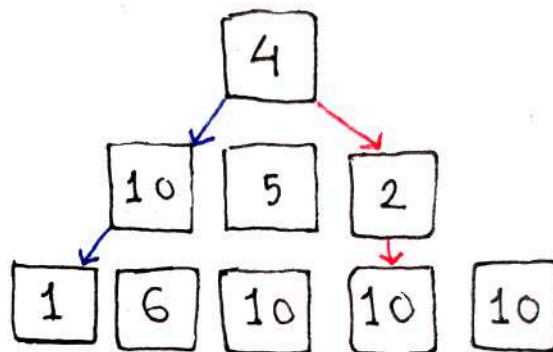
The seam carving process implemented is an iterative approach that will continue until the desired image dimensions are achieved.

Observation

Based on our observations, the edges of the energy map seem to have a value of 0, which may be caused by using the Sobel function to calculate the energy map. This resulted in the generated seam to consistently align with the edges, resulting in the seam carving process exclusively targeting the edges. Thus, we excluded the edges of the energy map while finding the seam, else the seam would always be generated at the edge of the energy map.

Comparison

Comparing both Dynamic programming and Greedy algorithm, Greedy algorithm is definitely faster as it does not require the computation of cumulative energy map. In terms of visual quality, the Greedy algorithm often leads to suboptimal results because it only considers local energy values, which may lead to seams disrupting the overall visual structure of the image. On the other hand, Dynamic programming considers the entire image and always produces a seam that minimises global disruption, thus preserving better visual quality.



<https://avikdas.com/2019/05/14/real-world-dynamic-programming-seam-carving.html>

Based on the diagram above showing the energy values, you can see that the red arrow represents the Greedy algorithm approach as compared to the blue arrow which represents the Dynamic programming approach. We can clearly see that the Greedy

algorithm does not work. Despite picking a lower energy value in the earlier row, it got stuck in a higher energy value region of the image.

Comparisons	Findings
Time Complexity	Although both the Dynamic Programming approach and Greedy Algorithm have the same simplified time complexity, the Greedy algorithm is slightly faster as it does not generate a cumulative map every iteration.
Image output	The Dynamic Programming approach tends to be slightly better . This could be due to how it finds the seam with the lowest possible values by considering the entire image unlike the Greedy Algorithm.

Original Image



Greedy Algorithm



Dynamic Programming



5. Overall Analysis and Discussion

Question 4

a) Time and space complexity

The function `CalculateEnergyMap` loops through all the pixels in the image, which means for a $W * H$ image, the time complexity for this function is $O(W * H)$. All functions following this will use the same W and H .

The function `CalculateCumMap` has a single for loop followed by two nested for loops. The single for loop loops through the last row of the energy map from the previous function and copies it over to the new cumulative map. The time complexity for looping through the whole last row is $O(W)$ as there are W pixels in a row. The next two nested for loops loop through all pixels of the image, once to calculate the cumulative value of the pixel, the next to normalise the values of the cumulative map. Each of those nested for loops have a time complexity of $O(W * H)$. This makes the overall time complexity for this function $O(W) + 2 * O(W * H)$ which simplifies to $O(W * H)$.

The function `FindVerticalSeamDP` loops through the first row of the cumulative map to find the smallest value and subsequently loops through the rows of the cumulative map to form a seam of lowest cumulative values. The first for loop has a time complexity of $O(W)$ and the second for loop has a time complexity of $O(H)$. The overall time complexity for this function is $O(W * H)$.

The function `VisualizeSeam` loops through all values in the seam which is equivalent to one column of pixels to show the seam that is being carved. The time complexity of this function is $O(H)$ as H is the number of rows the image has.

The last function `RemoveVerticalSeam` removes the seam that was found from the image. It loops through all rows in the image and shifts the image one column back to the column that the seam is at. The time complexity for the inner loop of column shifting is $O(W - n)$, n being the number of pixels that need to shift in the particular row. The overall time complexity of the function is then $O(H) * O(W - n)$ which simplifies to be $O(W * H)$.

Looking at the entire process of carving to the desired width, these functions will have to repeat n times with n being the number of vertical seams to be removed. Therefore the overall **time complexity** of this process will be $n * (O(W * H) + O(W * H) + O(W * H) + O(H) + O(W * H))$. Which turns out to be $4n * O(W * H) + n * O(H)$, which simplifies to be $O(W * H) + O(H)$. Which simplifies to be $O(W * H)$.

The whole process uses an energy map and a cumulative map of $W * H$ size. Therefore the **space complexity** would simplify to be $O(W * H)$.

b) Comparison with Greedy Algorithm

The function `FindVerticalSeamGreedy` goes through the last row of the cumulative map to find the column with the lowest energy value. Then, for the remaining rows, it checks the columns directly above, top-left (`startCol`), and top-right (`endCol`) of the previously recorded minimum location, then finds and records the new minimum location for each row.

The first check goes through the last row excluding the edges, thus it has a time complexity of $O(W-2)$. Then for the remaining checks the total time complexity is $O(3*H)$. Thus, the overall simplified time complexity for this function is $O(W) + O(H)$.

c) Any improvement, e.g., do we need to maintain the entire DP Table?

After removing a seam, the energy map is being recalculated for the entire image again. However, only the pixels neighbouring the removed seam need to be recalculated. An improvement that could be made would be to just recalculate the energy values for the affected pixels rather than the whole energy map.

6. Additional Features

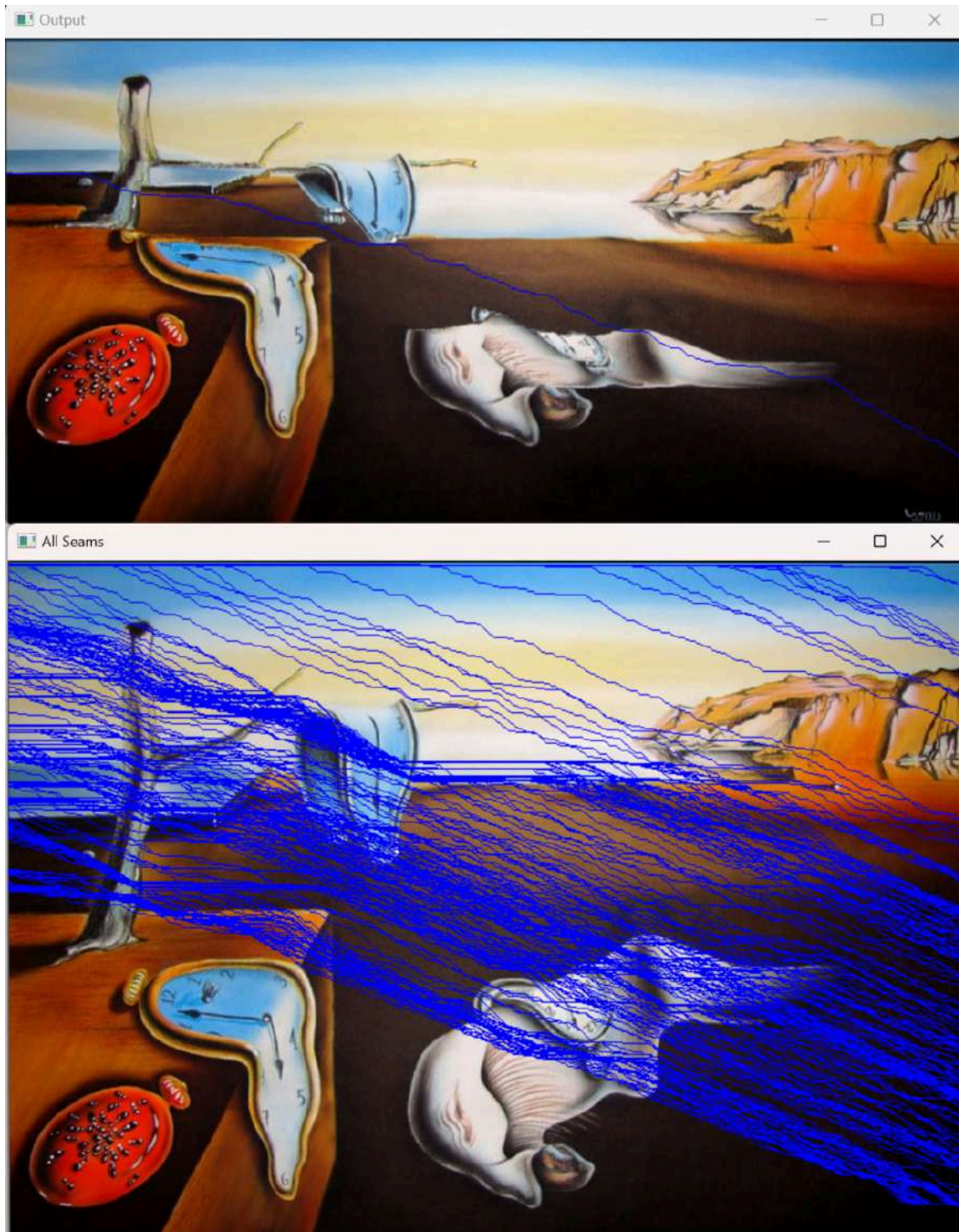
Question 5

a) Step by step visualisation, with all the seams visualised.

In the function `VisualizeVerticalSeam`, the function takes in 4 parameters, `cv::Mat& img` (the input image matrix to visualise the seam on), `std::vector<int> const& seam` (the vector of column indices for each row), `cv::Vec3b const& colour` (colour of the seam) as well as `int waitFMs` (the milliseconds to wait between the showing frames for the seams). The visualisation on the same picture will be different for both Dynamic Programming and Greedy algorithms. The photos attached below for the visualisations are based on the dynamic programming implementation.



In the function `VisualizeHorizontalSeam`, the function takes in 4 parameters, `cv::Mat& img` (the input image matrix to visualise the seam on), `std::vector<int> const& seam` (the vector of row indices for each column), `cv::Vec3b const& colour` (colour of the seam) as well as `int waitFMs` (the milliseconds to wait between the showing frames for the seams).



Our Visualisation makes use of:

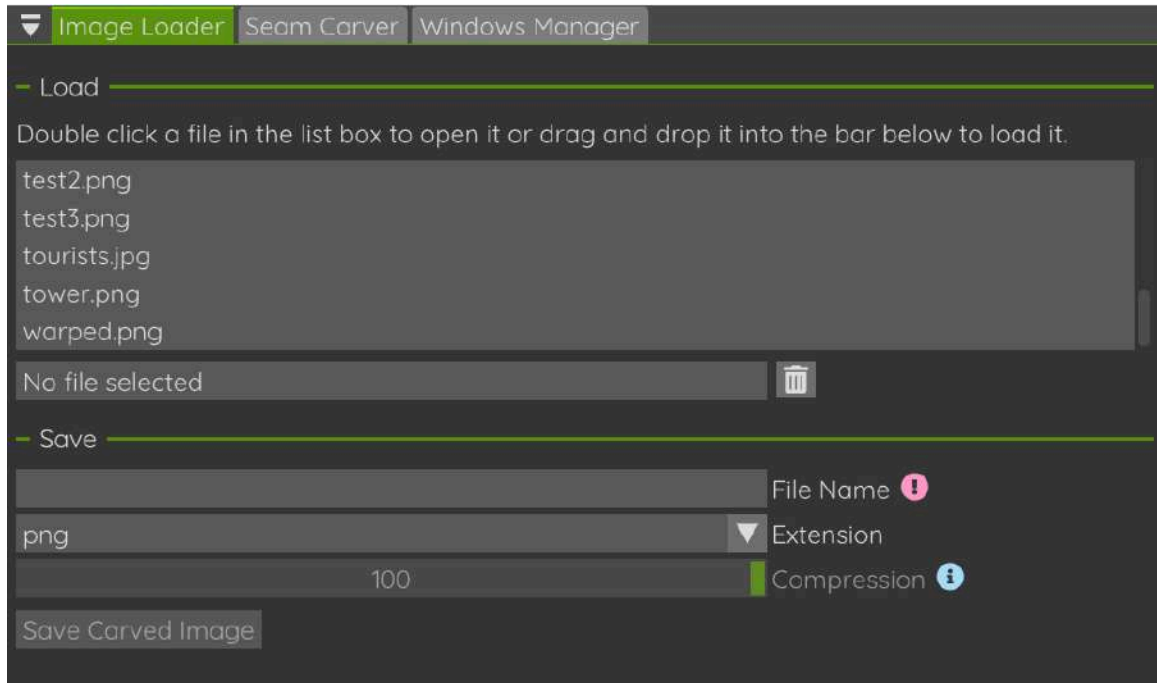
1. Output Window - Shows the current image with only the latest seam
2. All Seams Window - Shows all seams that have been found so far
3. Seams are coloured blue and appears as a red line path through the image

b) User interaction, enables users to interactively edit image sizes.

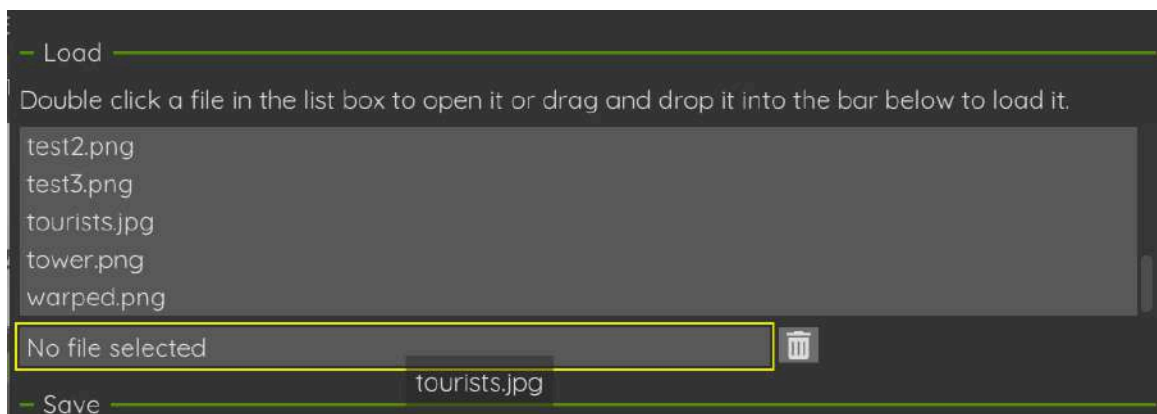
Our team has integrated the ImGui library to allow for a more user-friendly interface for users to select the image they would like to apply seam carving on, with various extra settings.

Image Loader Widget

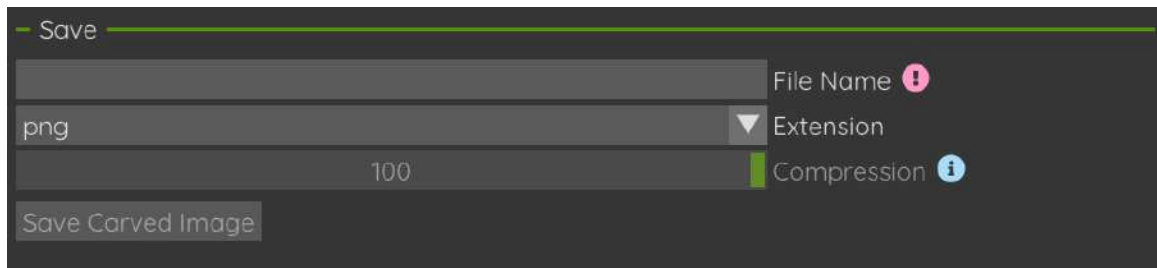
Upon Double Clicking on the list of png/jpg, a preview of the image will show up.



Users will be able to drag the png/jpg of their choice into this box to load it.



Users have the option to save the image after the carving process.

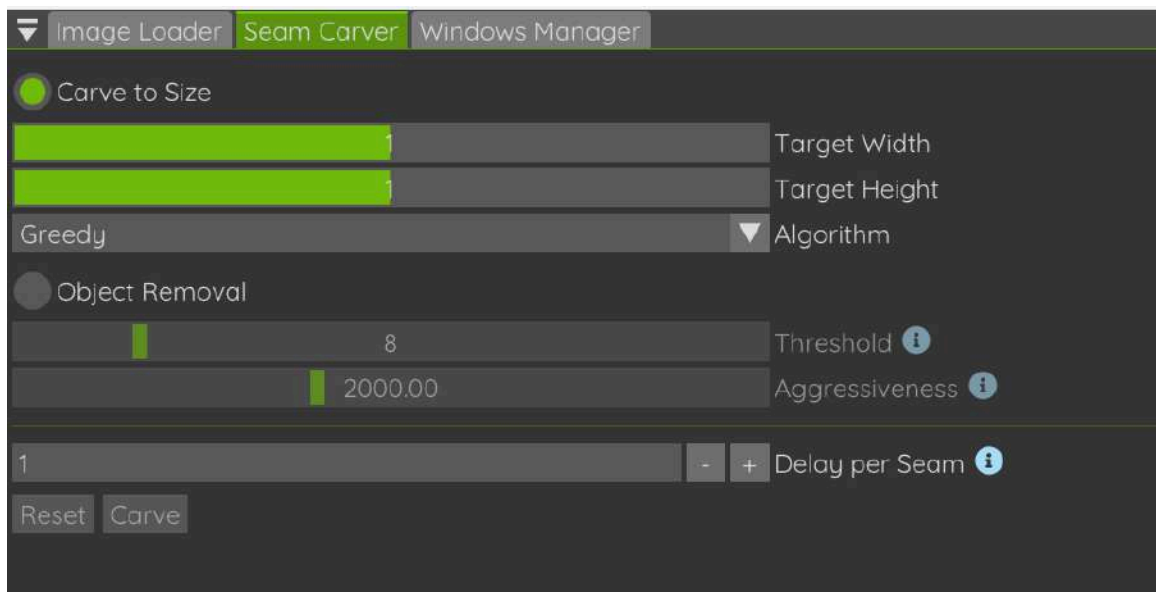


A dark-themed UI section titled "Save" with a green underline. It contains three input fields: a text field for "File Name" with a pink info icon, a dropdown menu for "Extension" currently showing "png", and a slider for "Compression" set to "100" with a green bar and a blue info icon. Below these fields is a button labeled "Save Carved Image".

Seam Carver Widget

Users will be able to adjust their desired target width and height which they would like their image to be carved by using the slider.

Similarly, users would also be able to adjust the threshold and aggressiveness values if they use the object removal settings. Threshold determines how much the area to be removed. The larger this value, the looser the bound of the removal area. Aggressiveness determines how much the algorithm prioritises the area to be removed. The higher the value, the more the algorithm prioritises this area to be removed.

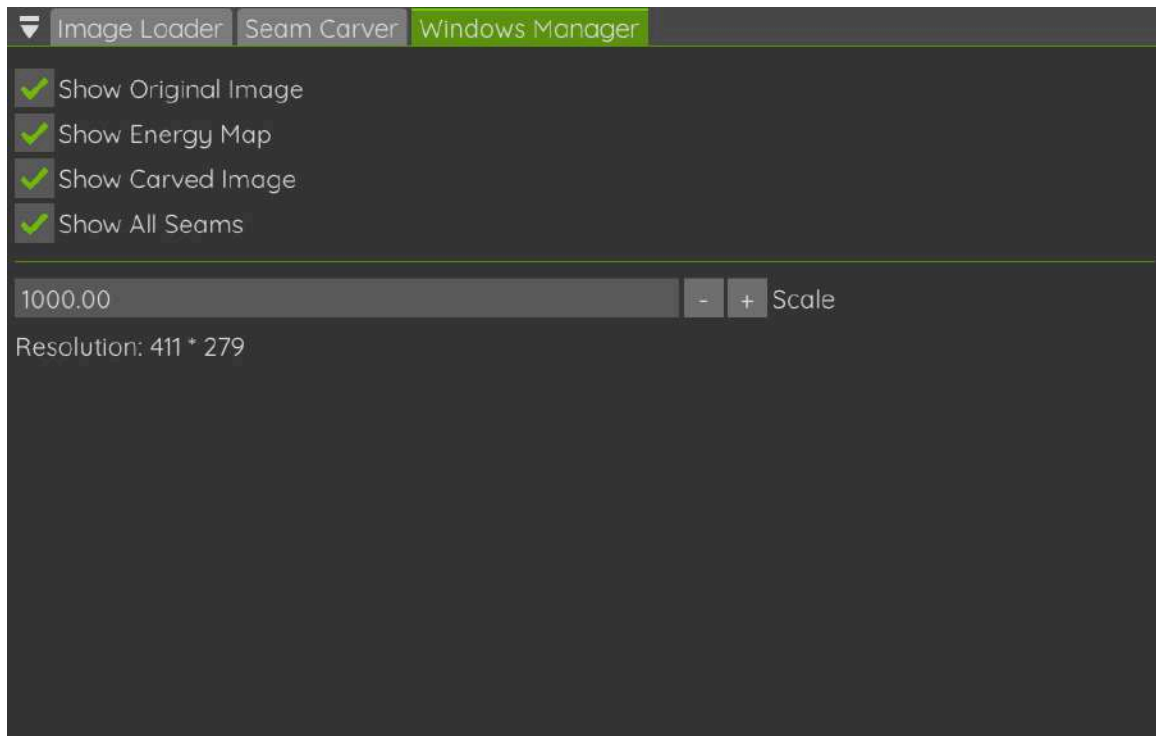


A dark-themed UI for the "Seam Carver" widget, with tabs for "Image Loader", "Seam Carver" (active), and "Windows Manager". It features two main sections: "Carve to Size" and "Object Removal". The "Carve to Size" section has two green sliders for "Target Width" and "Target Height", a dropdown for "Algorithm" set to "Greedy", and a "Delay per Seam" input set to "1" with minus and plus buttons. The "Object Removal" section has a slider for "Threshold" set to "8" and a slider for "Aggressiveness" set to "2000.00". Both sliders have green bars and blue info icons. At the bottom are "Reset" and "Carve" buttons.

Windows Manager Widget

We have a few windows settings such as:

1. Enable / Disable to show Original Image
2. Enable / Disable to show Carved Image
3. Enable / Disable to show All Seams
4. Enable / Disable to show Energy Map
5. Enable / Disable to show Energy Graph
6. Scale the Size of the Window



As an extra bonus, we have implemented content/object awareness removal through mouse-based outlining the object. The outlining is done by drawing circles on the mask at brush position and draws a red overlay similar to the software called JS Image Carver.

In the function `ContentAwareRemoval`, the function takes in the image matrix as a parameter and removes the user defined marked area using seam carving. The function starts by converting brush strokes to mask regions where it iterates through each row of the brush masked region and finds continuous masked regions in each row and adds the regions that need to be carved out from the image and stores that data in a vector (`std::vector<util::Mask> toRemove`).

Original Image:



Outlined Region (Object to be Removed):



Output Image (After Object Removal):

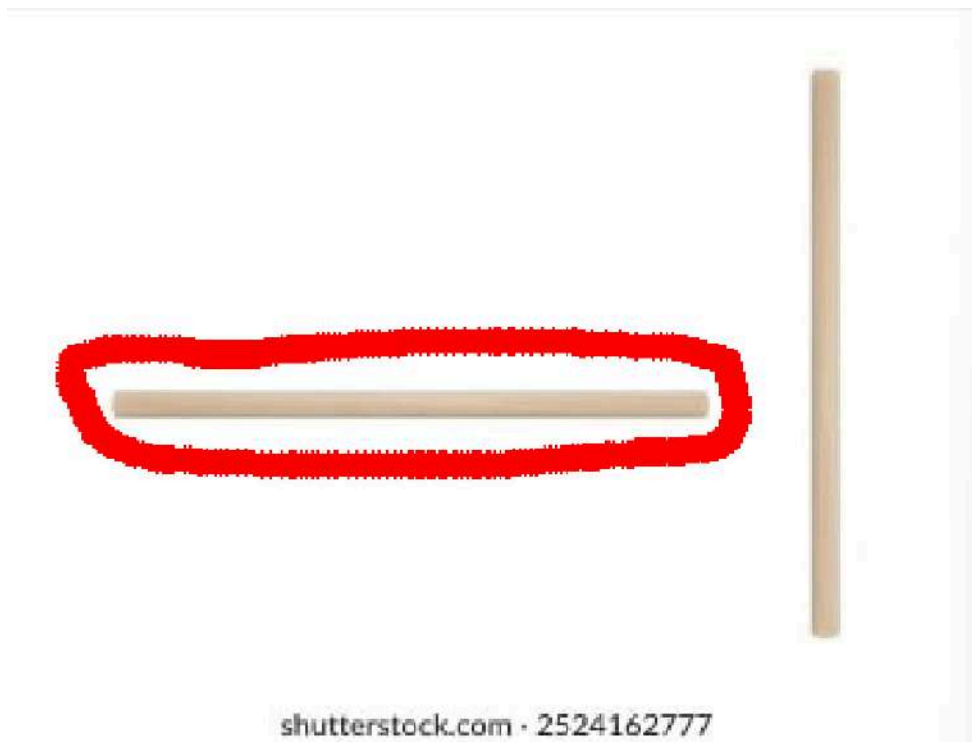


The special feature in this function is that it creates copies of the image and brush mask and then uses these copies to simulate both horizontal and vertical seam removal to determine which direction of seam removal is more efficient on the object outlined. To determine the efficiency of the direction, we count how many vertical and horizontal seams are needed to remove the selected area. The one with the fewer number of seams will be chosen as the efficient method to remove the area. It also makes use of the Dynamic Programming approach instead of the Greedy approach to search for the horizontal/vertical seam.

Original Image

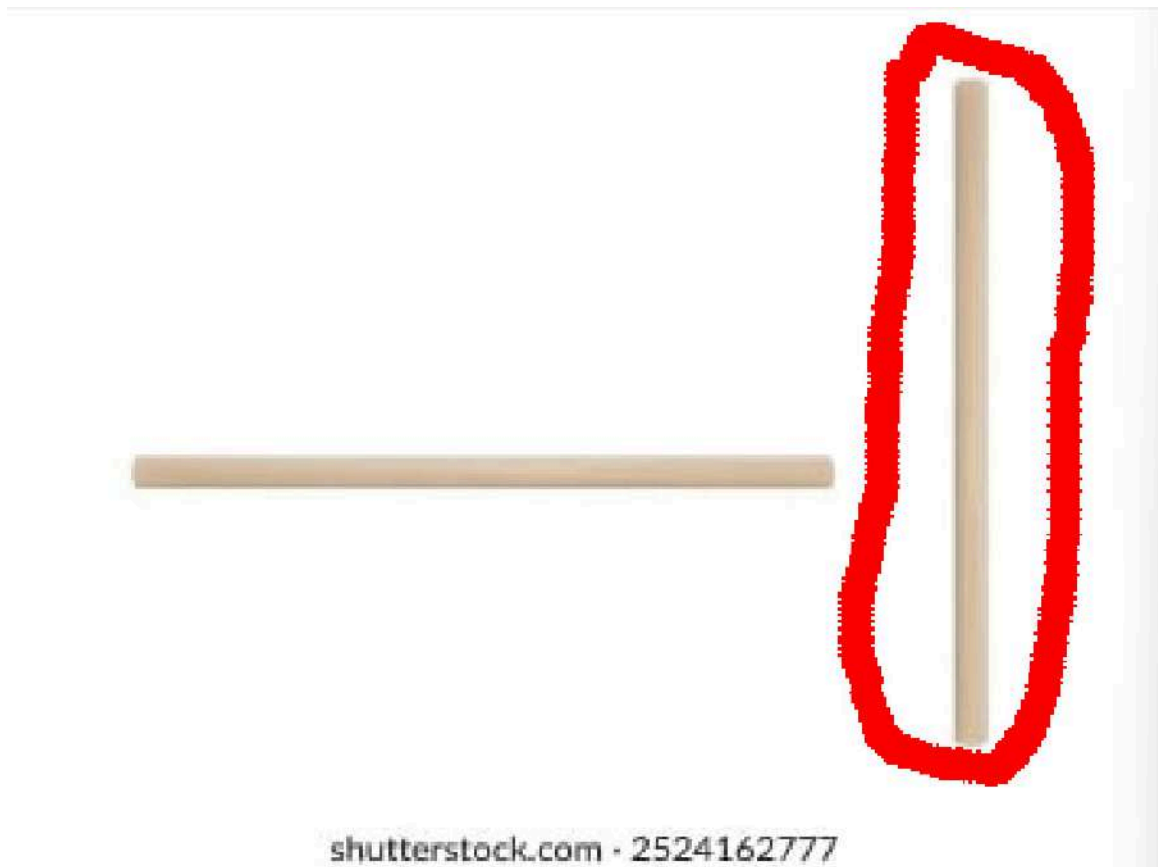


Horizontal Carving





Vertical Carving



Vertical Output



c) Implement using graph-cuts and compare with the Dynamic Programming approach

To give some background context, Graph-cuts in the context of image processing is a technique that treats an image as a graph where the pixels are considered nodes and the edges represent the relationship between the pixels. The algorithm finds the optimal seams by constructing a graph where the edge weights represent energy values between pixels and then find the optimal seam to remove based on the minimum cut.

There are many well known algorithms for graph-cut such as Ford-Fulkerson Algorithm, Edmonds-Karp Algorithm, and more. However, for our implementation, we used a library called maxflow that uses Boykov-Kolmogorov Algorithm to compute maxflow.

Based on our research, Boykov-Kolmogorov Algorithm is more suitable for graphs that tend to have a grid-link structure and are relatively sparse. This might be more appropriate for seam carving as our graph will be a grid.

In the function `FindVerticalSeamGraphCut`, the function takes in an energy map matrix and returns a seam path based on which nodes are in the sink segment. This function starts off by constructing a graph which represents each pixel and adds 3 types of edges for each node (pixel). The sum of the connected pixels are stored in the edges connecting the 2 nodes. After constructing the energy graph, it then connects the leftmost column, the source and the rightmost column to the sink and uses a function call in a library that makes use of Max-Flow Algorithm to find the minimum cut.

In the function `FindHorizontalSeamGraphCut`, it is similar to `FindVerticalSeamGraphCut`, where in this case, it iterates through rows instead of columns when connecting the adjacent pixels during the construction of the graph. In this context, it connects the top row to source and bottom row to sink. It then returns horizontal seam paths based on sink segment nodes which makes use of Max-Flow Algorithm to find the minimum cut as well.

In term of time complexity, the worst-case complexity would be $O(E * F)$ where E is the number of edges and F is the maximum flow (depends on number of pixels and the energy map).

Comparing Graph-Cuts with Dynamic Programming, graph cut is a global optimization approach that models the problem as a graph of nodes and edges aiming to partition the graph (image) into two regions and the region where the changes happen will be the seam to remove. Whereas for Dynamic Programming, it solves the problem locally by looking at optimal subproblems and making decisions based on previous solutions.

Feature	Graph Cut	Dynamic Programming (DP)
Time Complexity	$O(E * F)$	$O(W * H)$
Space Complexity	Requires a graph with nodes and edges	Require Cumulative map
Implementation Complexity	Requires graph class / structure and flow algorithms	Simple DP approach can be implemented

7. References

Libraries used:

1. OpenCV
 - a. Image processing
2. Matplotlib-cpp
 - a. For plotting of the energy map graph
3. Numpy (installed as part of Python 3.8.2)
 - a. For calculation of the energy map graph
4. Maxflow
 - a. For graph cut
5. ImGui
 - a. User controls (choose which method)
6. openGL
 - a. Needed for ImGui to run properly

Articles used for guidance:

Seam carving with dynamic programming approach

1. [Seam Carving with Dynamic Programming and Basic Energy Calculation | by Himanshu Sharma | Oct. 2024 | Medium](#)
2. <https://avikdas.com/2019/05/14/real-world-dynamic-programming-seam-carving.html>
3. [JS IMAGE CARVER](#)
4. <https://www.youtube.com/watch?v=rpB6zQNsbQU&t=760s>

Object removal using brush

1. [JS IMAGE CARVER](#)